**ΕΞΟΡΥΞΗ ΔΕΔΟΜΕΝΩΝ & ΜΗΧΑΝΙΚΗ ΜΑΘΗΣΗ**

# Αναφορά Εργασιας 2025-2026

*Μάιος 2026*

*Ομάδα :*


| ***Ονοματεπώνυμο*** | ***ΑΜ*** | ***Ετος*** | ***email*** |
|---|---|---|---|
| *Ιωάννης Κώτσαλος* | *1100603* | *4ο* | *st1100603@ceid.upatras.gr* |
| *Ανδρέας Καραμαλίκης* | *1100562* | *4ο* | *st1100562@ceid.upatras.gr* |

# Περιεχόμενα :

**Περιεχόμενα :** 2
**Μέρος 1ο: Διερευνητική Ανάλυση (EDA)** 3
1. Εισαγωγή και Φόρτωση Δεδομένων 3
2. Αρχική Στατιστική Ανάλυση (df.info / df.describe) 3
3. Καθαρισμός Δεδομένων 4
Αντικατάσταση Inf/NaN και Αφαίρεση Διπλότυπων 4
Αυστηρός Έλεγχος Αρνητικών Τιμών 4
4. Πίνακας Συσχέτισης και Επιλογή Χαρακτηριστικών 4
Heatmap Πριν τον Καθαρισμό 4
Αφαίρεση Πολυσυγγραμμικότητας - Highly Correlated Features (threshold = 0.90) 5
Heatmap Μετά την Επιλογή Χαρακτηριστικών 5
5. Ανάλυση Διακύμανσης μέσω Box Plots 6
Αθροιστικά Box Plots (Συνολική Εικόνα) 6
Έλεγχος Binary Flags 7
Αφαίρεση Μηδενικής Διακύμανσης 7
Feature vs Label Box Plots 8
Αφαίρεση Επικαλυπτόμενων Χαρακτηριστικών 8
6. Κατανομή Κλάσεων 9
**Μέρος 2ο: Data Preprocessing & Classification** 9
1. Data Preprocessing 10
Classification Dataset - Stratified Sampling (20% sampling) 10
Clustering Dataset (4% sampling) 10
2. Κωδικοποίηση Ετικετών και Εξαγωγή 10
Label Encoding 10
Εξαγωγή Αρχείων 10
3. Προετοιμασία Δεδομένων για Classification 10
Διαχωρισμός Train / Validation / Test Set 10
Κανονικοποίηση Χαρακτηριστικών 11
Αντιμετώπιση Ανισορροπίας Κλάσεων 11
4. Υλοποίηση Αλγορίθμων Classification 11
4.1 Logistic Regression 11
4.2 Decision Tree 12
4.3 Random Forest 12
5. Αξιολόγηση Μοντέλων 12
5.1 Logistic Regression 13
5.2 Decision Tree 15
5.3 Random Forest 17
5.4 Συγκριτική Ανάλυση 19
**Μέρος 3ο: Clustering** 19
1. Data Preprocessing : 19
2. Υλοποίηση Αλγορίθμων Clustering 20

K-Means..... 20
Ιεραρχική Συσταδοποίηση (Hierarchical Clustering)..... 22
DBSCAN..... 24
3. Αξιολόγηση και Συγκριτική Ανάλυση..... 25
**Περιβάλλον υλοποίησης..... 26**
1. Αναλυτική Καταγραφή Περιβάλλοντος Υλοποίησης..... 26
2. Βιβλιοθήκες Λογισμικού.....26
3. Διαδικασία Εγκατάστασης Τοπικού Περιβάλλοντος.....27

# Μέρος 1ο: Διερευνητική Ανάλυση (EDA)

---

## 1. Εισαγωγή και Φόρτωση Δεδομένων

Το dataset προήλθε από το Kaggle (chethuhn/network-intrusion-dataset) και περιλαμβάνει καταγραφές δικτυακής κυκλοφορίας για ανίχνευση εισβολών. Τα δεδομένα ήταν χωρισμένα σε 8 αρχεία CSV, τα οποία φορτώθηκαν με τη βιβλιοθήκη pandas και ενώθηκαν σε ένα ενιαίο DataFrame μέσω της pd.concat().

Κατά τη φόρτωση αφαιρέθηκαν περιττά κενά από τα ονόματα των στηλών (df.columns.str.strip()), ώστε να αποφευχθούν προβλήματα αναγνώρισης χαρακτηριστικών. Το τελικό αρχικό dataset περιείχε 2.830.743 εγγραφές και 79 χαρακτηριστικά, μαζί με τη στήλη Label που περιγράφει τον τύπο κυκλοφορίας.

## 2. Αρχική Στατιστική Ανάλυση (df.info / df.describe)

Πραγματοποιήθηκε αρχική διερεύνηση των δεδομένων με τις συναρτήσεις df.info() και df.describe(), ώστε να εξεταστούν οι τύποι δεδομένων, οι ελλιπείς τιμές και βασικά στατιστικά μεγέθη όπως μέση τιμή, τυπική απόκλιση και τεταρτημόρια.

# 3. Καθαρισμός Δεδομένων

## Αντικατάσταση Inf/NaN και Αφαίρεση Διπλότυπων

Οι τιμές Inf και -Inf αντικαταστάθηκαν με NaN, ενώ στη συνέχεια αφαιρέθηκαν οι γραμμές με ελλιπείς τιμές (dropna()). Επιπλέον, διαγράφηκαν πλήρως διπλότυπες εγγραφές (drop_duplicates()).

## Αυστηρός Έλεγχος Αρνητικών Τιμών

Αφαιρέθηκαν μη έγκυρες αρνητικές τιμές από τις αριθμητικές στήλες. Εξαιρέθηκαν οι στήλες Init_Win_bytes_forward, Init_Win_bytes_backward και min_seg_size_forward, όπου η τιμή -1 θεωρείται έγκυρη για UDP ροές.

Μετά τον καθαρισμό, το dataset περιείχε 2.517.883 εγγραφές και 79 στήλες.

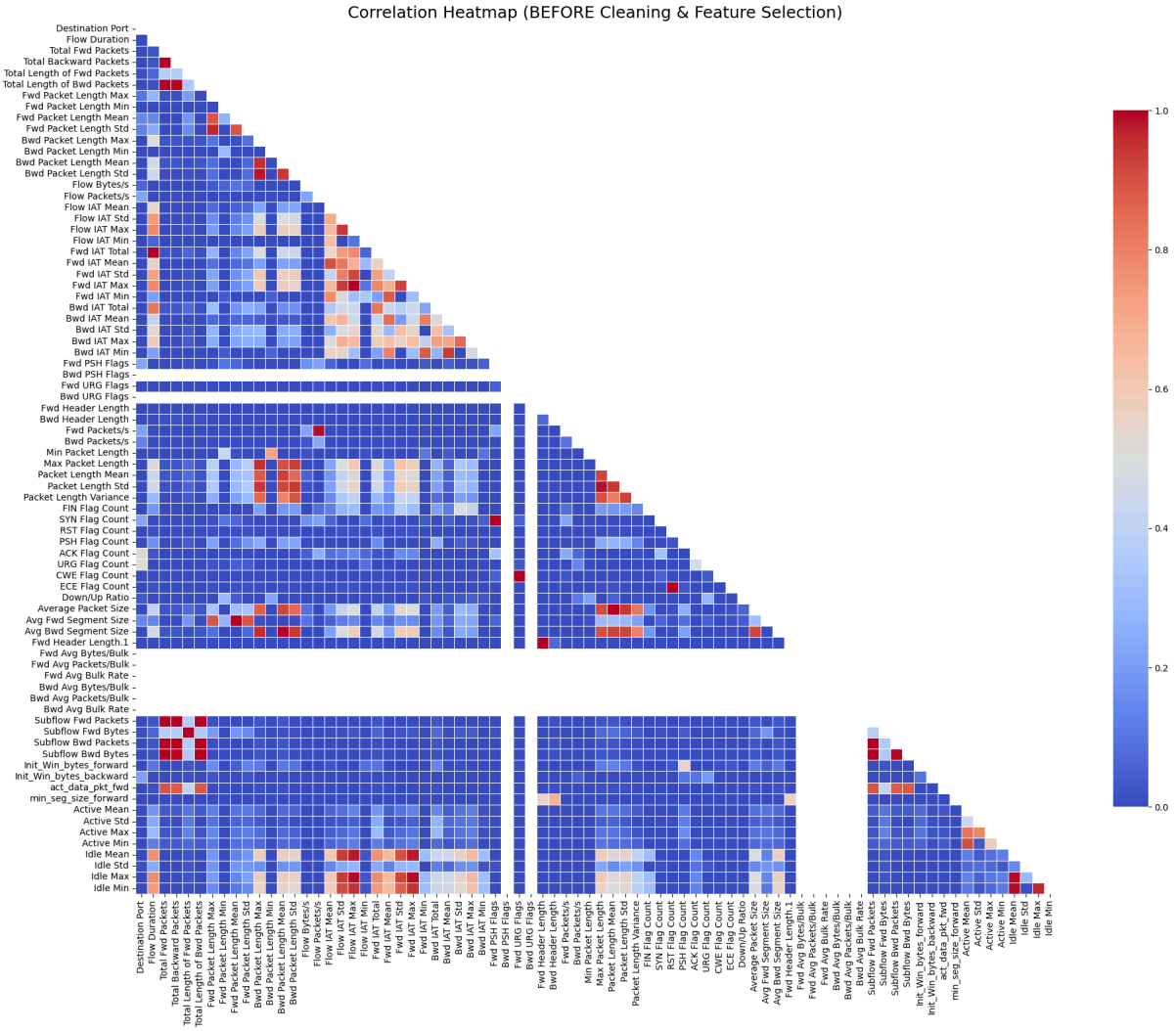
Οι στήλες με εντοπισμένες μη έγκυρες αρνητικές τιμές ήταν:

| Στήλη | Αρνητικές Εγγραφές |
|---|---|
| Flow Duration | 107 |
| Flow IAT Min | 2.773 |
| Fwd Header Length | 35 |

# 4. Πίνακας Συσχέτισης και Επιλογή Χαρακτηριστικών

## Heatmap Πριν τον Καθαρισμό

Δημιουργήθηκε heatmap του πίνακα συσχέτισης για την οπτικοποίηση των σχέσεων μεταξύ των χαρακτηριστικών. Χρησιμοποιήθηκε μόνο το κάτω τριγωνικό τμήμα του πίνακα ώστε να αποφευχθούν διπλές συγκρίσεις. Η ανάλυση έδειξε αρκετές ομάδες χαρακτηριστικών με πολύ υψηλή συσχέτιση.



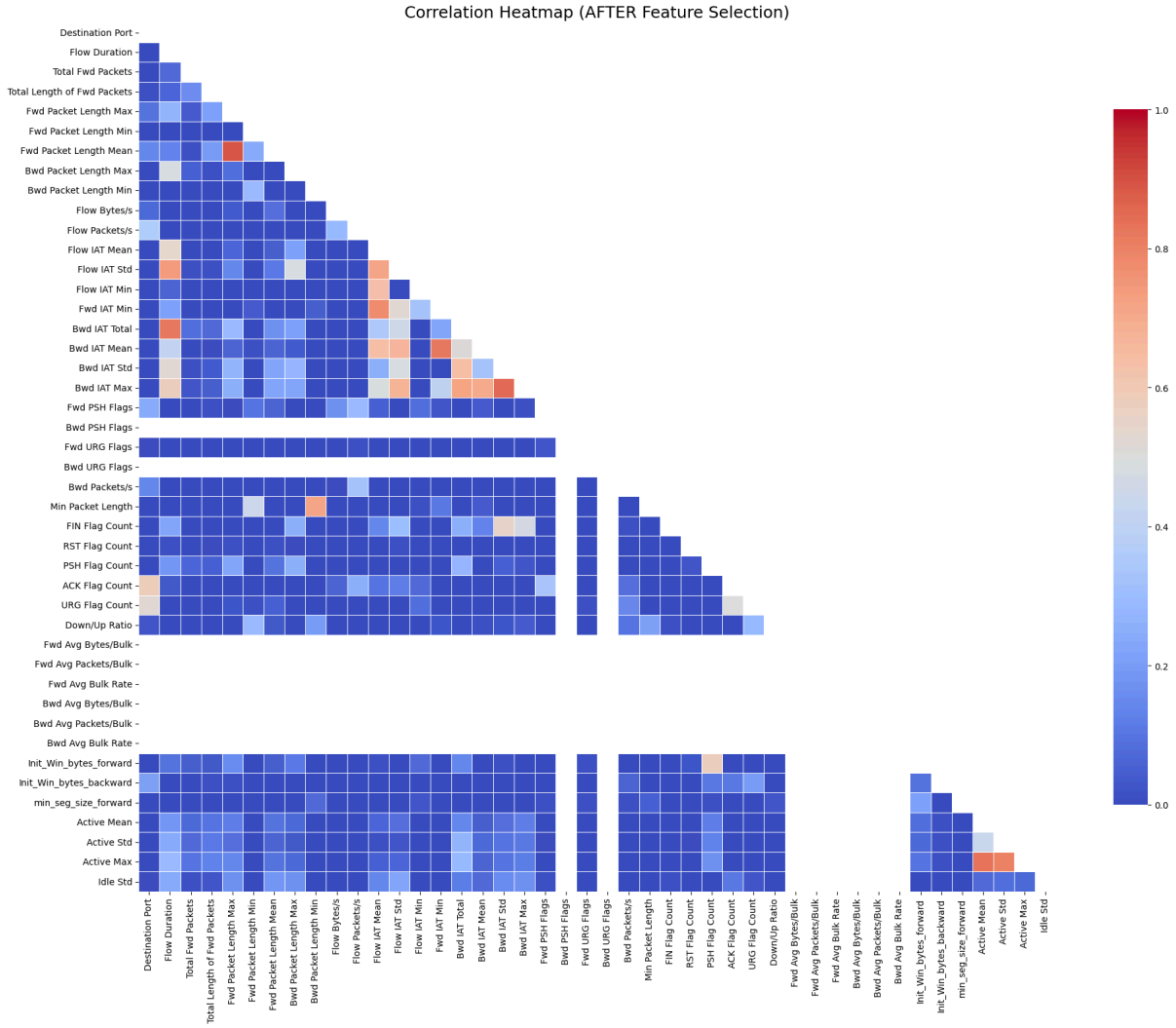

## Αφαίρεση Πολυσυγγραμμικότητας - Highly Correlated Features (threshold = 0.90)

Υπολογίστηκε ο πίνακας συσχέτισης Pearson για όλα τα αριθμητικά χαρακτηριστικά και εφαρμόστηκε κατώφλι συσχέτισης 0.90. Όταν δύο ή περισσότερα χαρακτηριστικά είχαν πολύ υψηλή συσχέτιση, διατηρούνταν το πρώτο και αφαιρούνταν τα υπόλοιπα, ώστε να μειωθεί η πολυσυγγραμμικότητα και η πλεονάζουσα πληροφορία.

Από τη διαδικασία **αφαιρέθηκαν 34 χαρακτηριστικά** και το dataset μειώθηκε από 79 σε 45 χαρακτηριστικά (συν τη στήλη Label), χωρίς απώλεια εγγραφών.

## Heatmap Μετά την Επιλογή Χαρακτηριστικών

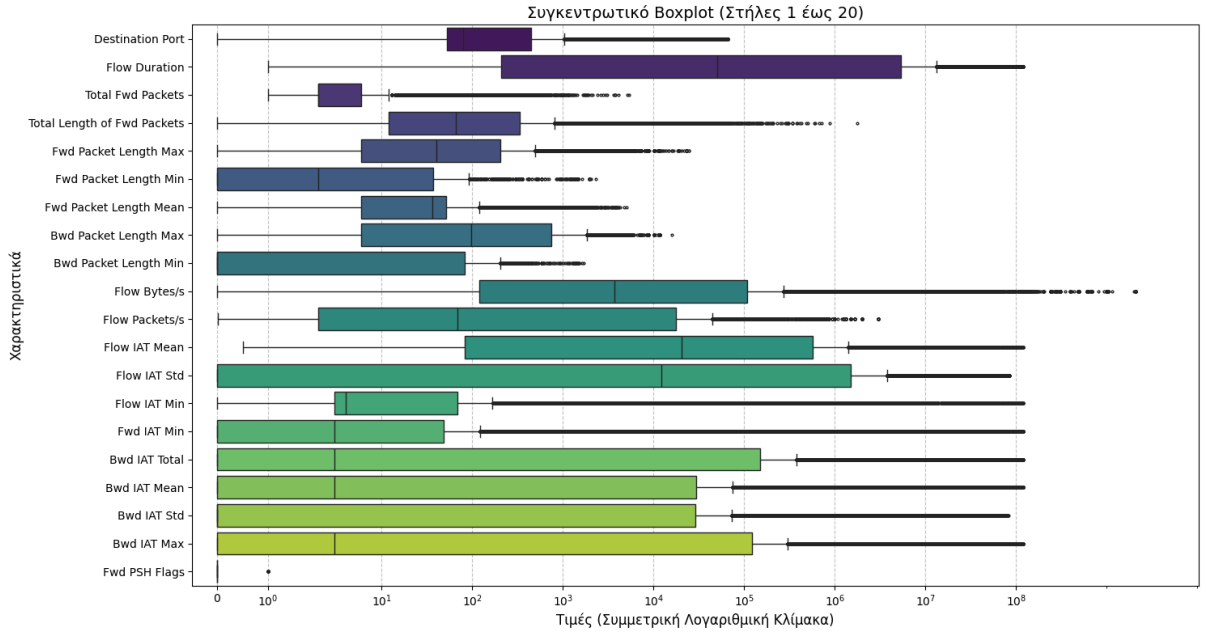
Δημιουργήθηκε δεύτερο heatmap μετά την επιλογή χαρακτηριστικών, το οποίο επιβεβαίωσε ότι οι περισσότερες πολύ υψηλές συσχετίσεις είχαν απομακρυνθεί.




# 5. Ανάλυση Διακύμανσης μέσω Box Plots

## Αθροιστικά Box Plots (Συνολική Εικόνα)

Δημιουργήθηκαν box plots για όλα τα αριθμητικά χαρακτηριστικά με σκοπό τον εντοπισμό features με πολύ μικρή διακύμανση ή περιορισμένο εύρος τιμών. Για ταχύτερη επεξεργασία χρησιμοποιήθηκε τυχαίο δείγμα 500.000 εγγραφών




*Ενδεικτικό boxplot για fatures 1-20*

## Έλεγχος Binary Flags

Κατά την εξέταση των αθροιστικών box plots εντοπίστηκαν 5 δυαδικές (binary) flag στήλες με εξαιρετικά χαμηλή διακύμανση: Fwd PSH Flags, Fwd URG Flags, FIN Flag Count, RST Flag Count και URG Flag Count. Για κάθε μία δημιουργήθηκε cross-tabulation με τη στήλη Label, ώστε να εκτιμηθεί αν παρέχουν διακριτική πληροφορία ανά κατηγορία επίθεσης.

Βάσει αυτής της ανάλυσης λήφθηκαν οι εξής αποφάσεις:

- **Μη χρήσιμα flags (αφαιρέθηκαν):** 'Fwd URG Flags' και 'RST Flag Count' — παρουσίαζαν μηδενική ή αμελητέα διακύμανση σε όλες τις κλάσεις και δεν πρόσφεραν διακριτική ισχύ.
- **«Χρυσά» flags (διατηρήθηκαν):** 'FIN Flag Count', 'URG Flag Count' και 'Fwd PSH Flags' — παρά τη χαμηλή τους διακύμανση, η cross-tabulation ανά Label αποκάλυψε ότι παρουσιάζουν σημαντικές διαφορές μεταξύ τύπων επιθέσεων και κανονικής κυκλοφορίας, άρα διατηρήθηκαν.

## Αφαίρεση Μηδενικής Διακύμανσης

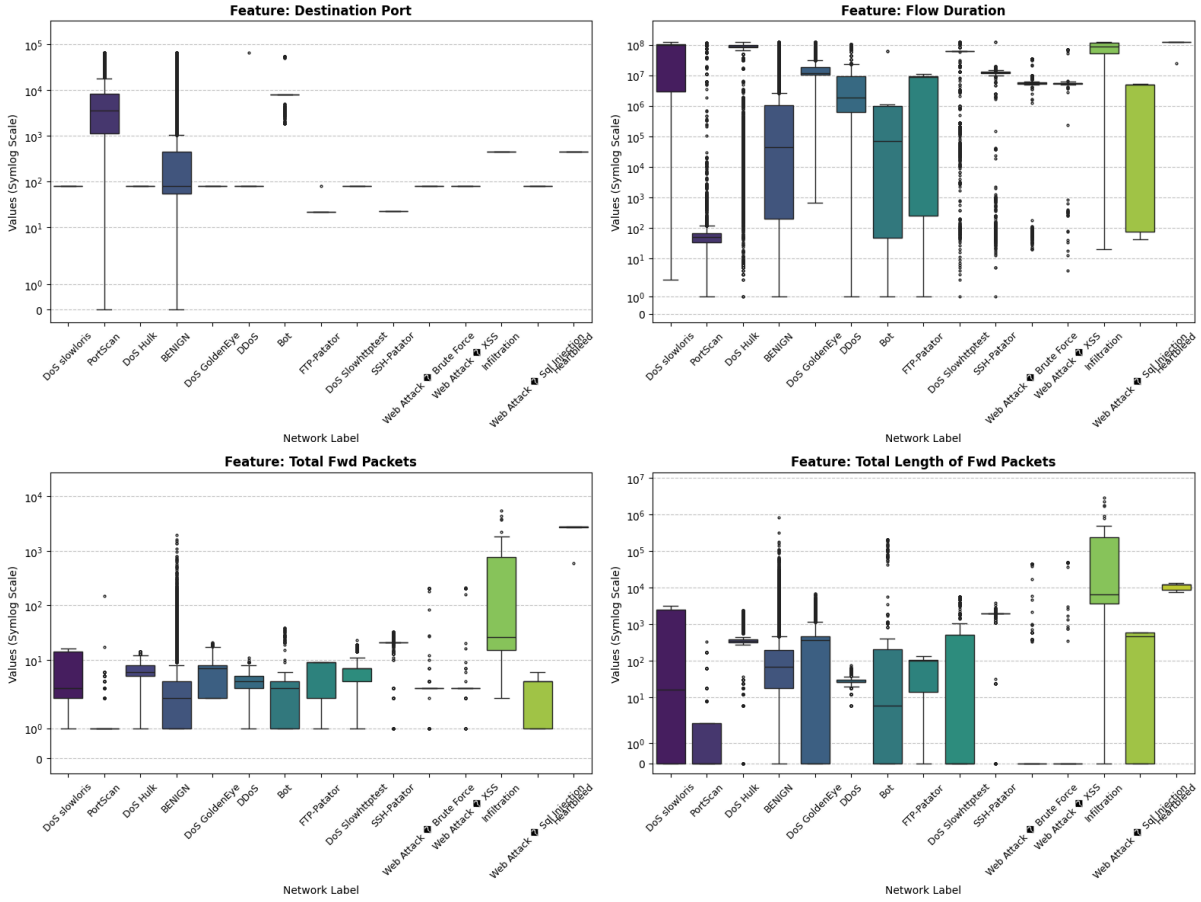
Εφαρμόστηκε ο αλγόριθμος VarianceThreshold, ο οποίος εντόπισε και αφαίρεσε 8 χαρακτηριστικά με μηδενική διακύμανση, δηλαδή στήλες με σταθερές τιμές σε όλες τις εγγραφές.

## Feature vs Label Box Plots

Στη συνέχεια δημιουργήθηκαν box plots ανά χαρακτηριστικό και ανά κατηγορία Label, ώστε να εξεταστεί η ικανότητα κάθε feature να διαχωρίζει τις διαφορετικές κλάσεις. Για τη διαδικασία χρησιμοποιήθηκε δειγματοληψία με ανώτατο όριο 40.000 εγγραφών ανά κλάση για ταχύτερη οπτικοποίηση.

Σε κάθε box plot αξιολογήθηκαν δύο κριτήρια για απόρριψη ενός χαρακτηριστικού:

- **Επικάλυψη IQR:** Όταν το διατεταρτημορικό εύρος (IQR) των boxplots επικαλυπτόταν εκτεταμένα σε όλες ή τις περισσότερες κλάσεις, το χαρακτηριστικό κρίνεται ανίκανο να διακρίνει μεταξύ κατηγοριών.
- **Ομοιομορφία διαμέσων (Median):** Όταν η γραμμή διαμέσου παρέμενε σχεδόν ίδια σε πολλές κλάσεις, το χαρακτηριστικό δεν έχει διακριτική αξία — ενδεχομένως θα μπέρδευε το μοντέλο προσφέροντας την ίδια κατανομή τιμών για διαφορετικές ετικέτες.




*Ενδεικτικό boxplot value vs labels per feature*

## Αφαίρεση Επικαλυπτόμενων Χαρακτηριστικών

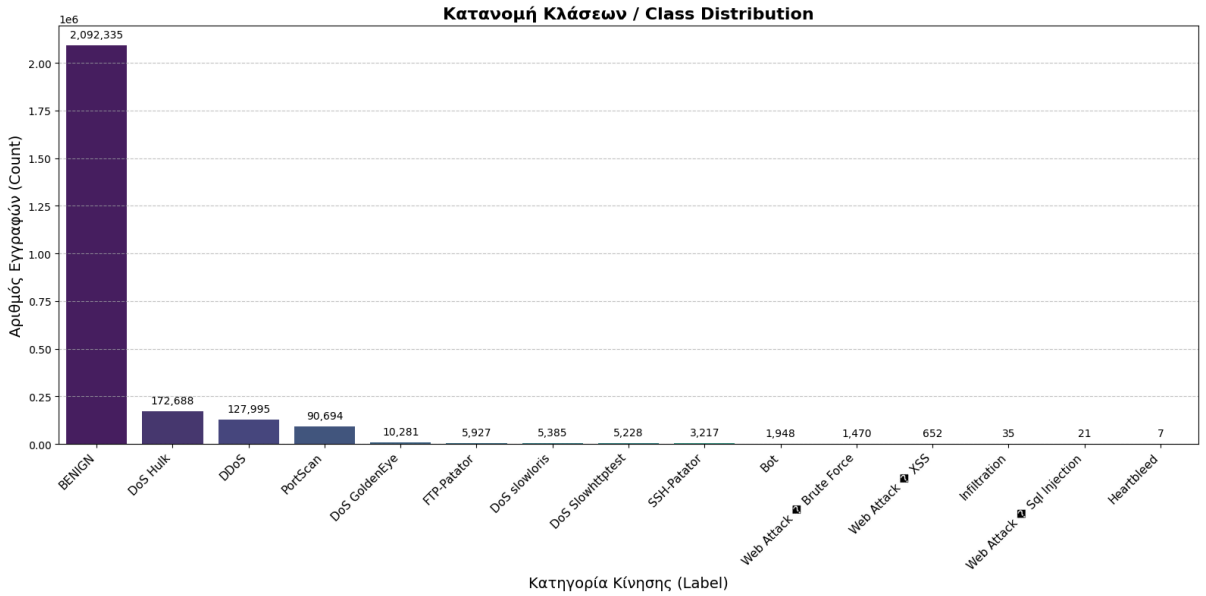
Βάσει της ανάλυσης των feature-vs-label box plots, αφαιρέθηκαν χειροκίνητα 9 χαρακτηριστικά που παρουσίαζαν εκτεταμένη επικάλυψη boxplots ή ομοιόμορφη κατανομή μεταξύ κλάσεων:

| |
|---|
| Fwd Packet Length Min |
| Bwd Packet Length Min |
| Flow IAT Min |
| PSH Flag Count |
| ACK Flag Count |
| Down/Up Ratio |
| min_seg_size_forward |
| Active Std |
| Min Packet Length |

Μετά από όλα τα βήματα feature selection, το τελικό καθαρό dataset αποτελούνταν από 26 στήλες και εξήχθη ως αρχείο Parquet (network_intrusion_cleaned.parquet).

# 6. Κατανομή Κλάσεων

Πριν από την εξαγωγή, δημιουργήθηκε ραβδόγραμμα (bar chart) για την οπτικοποίηση της κατανομής κλάσεων στο καθαρό dataset. Το γράφημα αποκάλυψε έντονη ανισορροπία κλάσεων (class imbalance): η κατηγορία BENIGN αντιπροσώπευε το 83,1% του συνόλου, ενώ τρεις κατηγορίες (Heartbleed, Web Attack–SQL Injection, Infiltration) είχαν μόλις 7, 21 και 35 εγγραφές αντίστοιχα.




# Μέρος 2ο: Data Preprocessing & Classification

# 1. Data Preprocessing

## Classification Dataset - Stratified Sampling (20% sampling)

Λόγω του μεγάλου μεγέθους του dataset, εφαρμόστηκε στρωματοποιημένη δειγματοληψία 20% για τη δημιουργία ενός πιο διαχειρίσιμου και αντιπροσωπευτικού συνόλου δεδομένων για classification.

Οι πολύ μικρές κλάσεις (Heartbleed, Web Attack–SQL Injection, Infiltration) διατηρήθηκαν πλήρως ώστε να μην χαθεί πληροφορία από σπάνιες επιθέσεις, ενώ οι υπόλοιπες κλάσεις δειγματοληπτήθηκαν διατηρώντας τις αρχικές αναλογίες τους.

Το τελικό dataset classification περιείχε 503.627 εγγραφές.

## Clustering Dataset (4% sampling)

Για τις ανάγκες clustering δημιουργήθηκε μικρότερο dataset με δειγματοληψία 4%, ακολουθώντας την ίδια λογική διατήρησης των σπάνιων κλάσεων.

Το τελικό dataset clustering περιείχε 20.206 εγγραφές.

# 2. Κωδικοποίηση Ετικετών και Εξαγωγή

## Label Encoding

Οι κατηγορίες της στήλης Label κωδικοποιήθηκαν αριθμητικά με χρήση του LabelEncoder της scikit-learn. Η αντιστοίχιση αποθηκεύτηκε σε αρχείο label_mapping.json, ώστε να είναι δυνατή η αποκωδικοποίηση των αποτελεσμάτων των μοντέλων.

## Εξαγωγή Αρχείων

Εξήχθησαν τα εξής τρία αρχεία για χρήση στα επόμενα μέρη της εργασίας:

- **network_intrusion_cleaned.parquet:** 2.517.883 εγγραφές, 26 στήλες — το πλήρες καθαρό dataset μετά από EDA.
- **network_intrusion_sampled.parquet:** 503.627 εγγραφές — το κύριο dataset για classification με αριθμητικές ετικέτες.
- **network_intrusion_clustering.parquet:** 20.206 εγγραφές — το μικρό dataset για clustering.
- **label_mapping.json:** Αντιστοίχιση ονομάτων κλάσεων → αριθμητικών κωδικών για αποκωδικοποίηση αποτελεσμάτων.

# 3. Προετοιμασία Δεδομένων για Classification

## Διαχωρισμός Train / Validation / Test Set

Το dataset network_intrusion_sampled.parquet χωρίστηκε σε σύνολα εκπαίδευσης, επικύρωσης και ελέγχου (train/validation/test) με στρωματοποιημένη δειγματοληψία (stratify=y), ώστε να διατηρηθούν οι αναλογίες των κλάσεων σε όλα τα υποσύνολα.

Ο διαχωρισμός πραγματοποιήθηκε με σταθερό random_state=42 για αναπαραγωγιμότητα των αποτελεσμάτων.

### Κανονικοποίηση Χαρακτηριστικών

Για τους αλγορίθμους που επηρεάζονται από την κλίμακα των δεδομένων, εφαρμόστηκε κανονικοποίηση μέσω `StandardScaler`. Η διαδικασία πραγματοποιήθηκε μόνο στα δεδομένα εκπαίδευσης και στη συνέχεια εφαρμόστηκε στα validation και test sets.

### Αντιμετώπιση Ανισορροπίας Κλάσεων

Καθώς το dataset παρουσιάζει σημαντική ανισορροπία μεταξύ φυσιολογικής και κακόβουλης κυκλοφορίας, χρησιμοποιήθηκε η επιλογή class_weight='balanced' στους αλγορίθμους που την υποστηρίζουν, ώστε να δοθεί μεγαλύτερη βαρύτητα στις μειοψηφικές κλάσεις.

## 4. Υλοποίηση Αλγορίθμων Classification

Για την ανίχνευση και ταξινόμηση των επιθέσεων υλοποιήθηκαν οι αλγόριθμοι Logistic Regression, Decision Tree και Random Forest.

Κάθε αλγόριθμος αξιολογήθηκε με δύο διαφορετικές προσεγγίσεις:

- κλασικό διαχωρισμό train/validation/test,
- και 5-Fold Cross Validation, ώστε να εξεταστεί η σταθερότητα και η ικανότητα γενίκευσης των μοντέλων σε διαφορετικά υποσύνολα δεδομένων.

Για τη βελτιστοποίηση των υπερπαραμέτρων χρησιμοποιήθηκε GridSearchCV σε συνδυασμό με StratifiedKFold.

### 4.1 Logistic Regression

Ο αλγόριθμος Logistic Regression χρησιμοποιήθηκε ως baseline μοντέλο ταξινόμησης.
Εφαρμόστηκε StandardScaler στα δεδομένα και πραγματοποιήθηκε βελτιστοποίηση υπερπαραμέτρων μέσω GridSearchCV, με δοκιμή διαφορετικών τιμών για:

- C
- solver
- penalty

Η αξιολόγηση έγινε με διασταυρωμένη επικύρωση (StratifiedKFold).

### 4.2 Decision Tree

Για το μοντέλο Decision Tree πραγματοποιήθηκε αναζήτηση υπερπαραμέτρων μέσω GridSearchCV, εξετάζοντας:

- max_depth
- min_samples_split
- min_samples_leaf
- criterion

Η αξιολόγηση έγινε με train/validation split και με 5-Fold Cross Validation, με στόχο τον περιορισμό του overfitting και τη βελτίωση της γενίκευσης.

### 4.3 Random Forest

Ο αλγόριθμος Random Forest εφαρμόστηκε ως ensemble μέθοδος βασισμένη σε πολλαπλά δέντρα απόφασης.
Χρησιμοποιήθηκε class_weight='balanced' για καλύτερη διαχείριση της ανισορροπίας των κλάσεων.

Πραγματοποιήθηκε tuning των παραμέτρων:

- n_estimators
- max_depth
- min_samples_split
- min_samples_leaf

και το μοντέλο αξιολογήθηκε τόσο με train/validation/test split όσο και με 5-Fold Cross Validation.

## 5. Αξιολόγηση Μοντέλων

Η αξιολόγηση των μοντέλων πραγματοποιήθηκε στο test set με χρήση των μετρικών:

- Accuracy
- Precision
- Recall

- F1-score

Επιπλέον, δημιουργήθηκαν confusion matrices και classification reports για την αναλυτική αξιολόγηση της απόδοσης κάθε μοντέλου στις διαφορετικές κατηγορίες επιθέσεων.

## 5.1 Logistic Regression

Το Logistic Regression παρουσίασε τη χαμηλότερη συνολική απόδοση από τους τρεις αλγορίθμους. Παρότι πέτυχε υψηλό Recall (~0.87–0.88), το Precision παρέμεινε ιδιαίτερα χαμηλό (~0.31–0.32), γεγονός που δείχνει ότι το μοντέλο παρήγαγε μεγάλο αριθμό false positives.

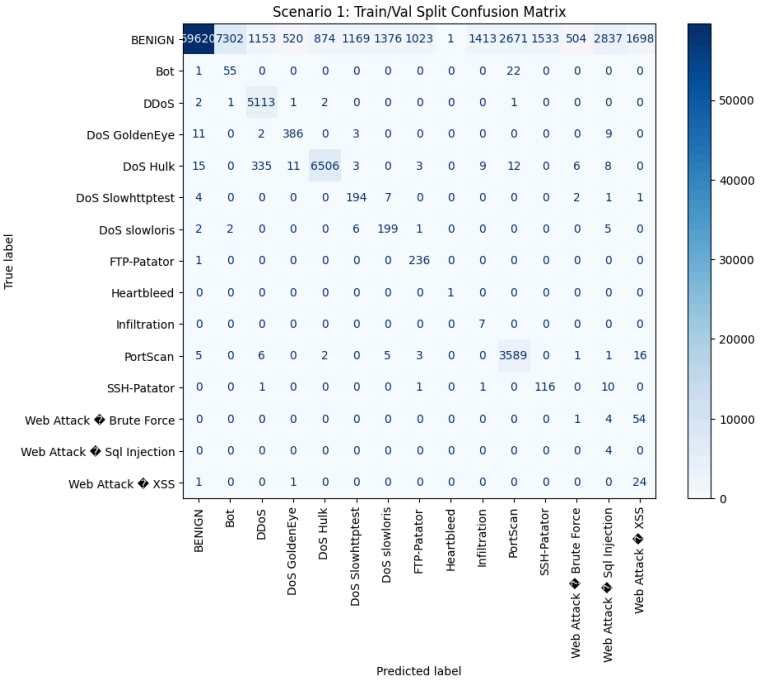
```
============================================================
EVALUATING Logistic Regression (Train/Validation Split Model)
============================================================
                           precision    recall  f1-score   support

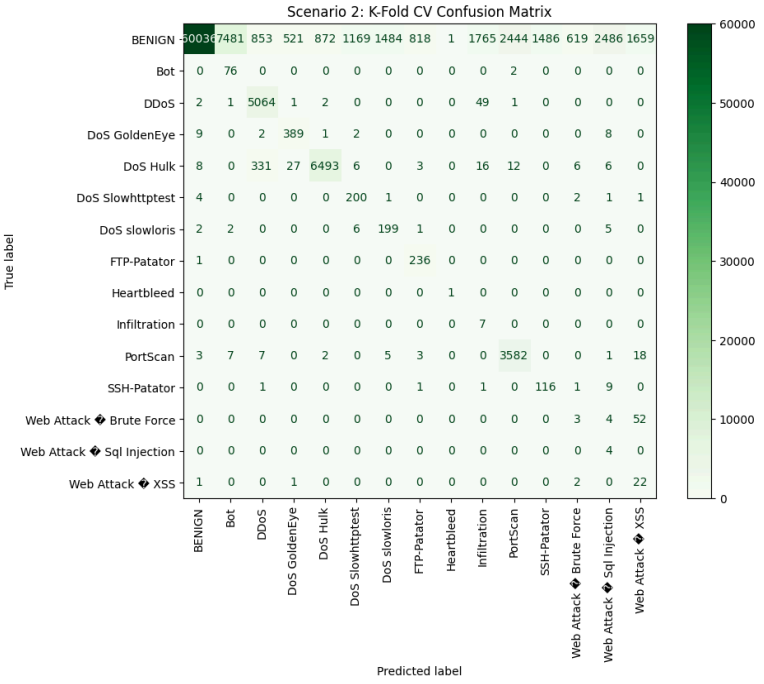
                   BENIGN       1.00      0.71      0.83     83694
                      Bot       0.01      0.71      0.01        78
                     DDoS       0.77      1.00      0.87      5120
            DoS GoldenEye       0.42      0.94      0.58       411
                 DoS Hulk       0.88      0.94      0.91      6908
         DoS Slowhttptest       0.14      0.93      0.24       209
            DoS slowloris       0.13      0.93      0.22       215
              FTP-Patator       0.19      1.00      0.31       237
               Heartbleed       0.50      1.00      0.67         1
             Infiltration       0.00      1.00      0.01         7
                 PortScan       0.57      0.99      0.72      3628
              SSH-Patator       0.07      0.90      0.13       129
  Web Attack  Brute Force       0.00      0.02      0.00        59
Web Attack  Sql Injection       0.00      1.00      0.00         4
          Web Attack  XSS       0.01      0.92      0.03        26

                 accuracy                           0.76    100726
                macro avg       0.31      0.87      0.37    100726
             weighted avg       0.95      0.76      0.83    100726
```

```
============================================================
EVALUATING Logistic Regression (K-Fold Cross-Validation Model)
============================================================
```

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| BENIGN | 1.00 | 0.72 | 0.84 | 83694 |
| Bot | 0.01 | 0.97 | 0.02 | 78 |
| DDoS | 0.81 | 0.99 | 0.89 | 5120 |
| DoS GoldenEye | 0.41 | 0.95 | 0.58 | 411 |
| DoS Hulk | 0.88 | 0.94 | 0.91 | 6908 |
| DoS Slowhttptest | 0.14 | 0.96 | 0.25 | 209 |
| DoS slowloris | 0.12 | 0.93 | 0.21 | 215 |
| FTP-Patator | 0.22 | 1.00 | 0.36 | 237 |
| Heartbleed | 0.50 | 1.00 | 0.67 | 1 |
| Infiltration | 0.00 | 1.00 | 0.01 | 7 |
| PortScan | 0.59 | 0.99 | 0.74 | 3628 |
| SSH-Patator | 0.07 | 0.90 | 0.13 | 129 |
| Web Attack  Brute Force | 0.00 | 0.05 | 0.01 | 59 |
| Web Attack  Sql Injection | 0.00 | 1.00 | 0.00 | 4 |
| Web Attack  XSS | 0.01 | 0.85 | 0.02 | 26 |
| accuracy | | | 0.76 | 100726 |
| macro avg | 0.32 | 0.88 | 0.38 | 100726 |
| weighted avg | 0.96 | 0.76 | 0.83 | 100726 |



Η πολύ μικρή διαφορά μεταξύ split και cross-validation δείχνει ότι το μοντέλο ήταν σταθερό, αλλά συνολικά αδύναμο ως προς τη διακριτική ικανότητα. Η συμπεριφορά αυτή οφείλεται κυρίως στη γραμμική φύση του Logistic Regression, το οποίο δυσκολεύεται να μοντελοποιήσει τις πολύπλοκες μη γραμμικές σχέσεις και αλληλεπιδράσεις μεταξύ των χαρακτηριστικών του δικτυακού traffic.

### 5.2 Decision Tree

Το Decision Tree παρουσίασε σημαντικά καλύτερη απόδοση σε σχέση με το Logistic Regression, αλλά και μεγάλη διαφορά μεταξύ των δύο προσεγγίσεων αξιολόγησης.

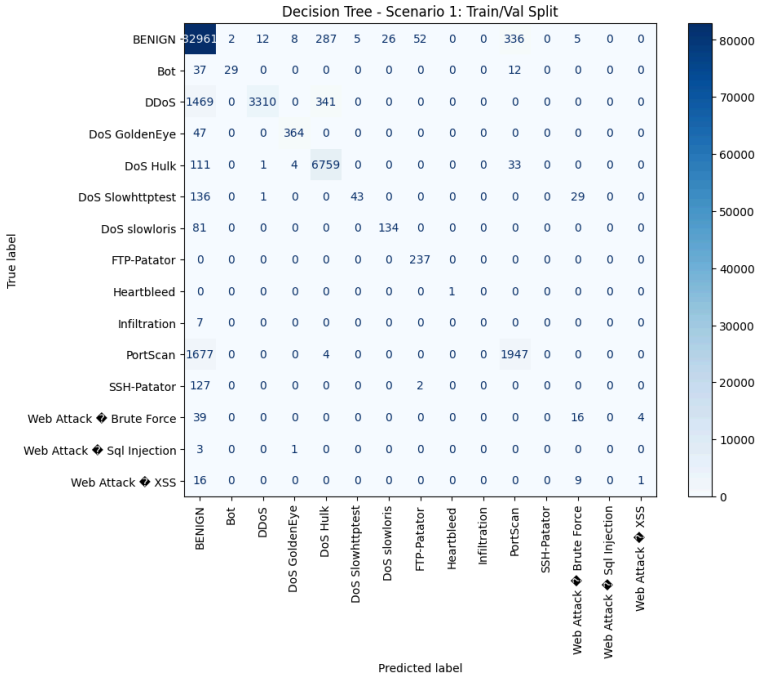
```
============================================================
EVALUATING DECISION TREE (Train/Val Split)
============================================================
                           precision    recall  f1-score   support

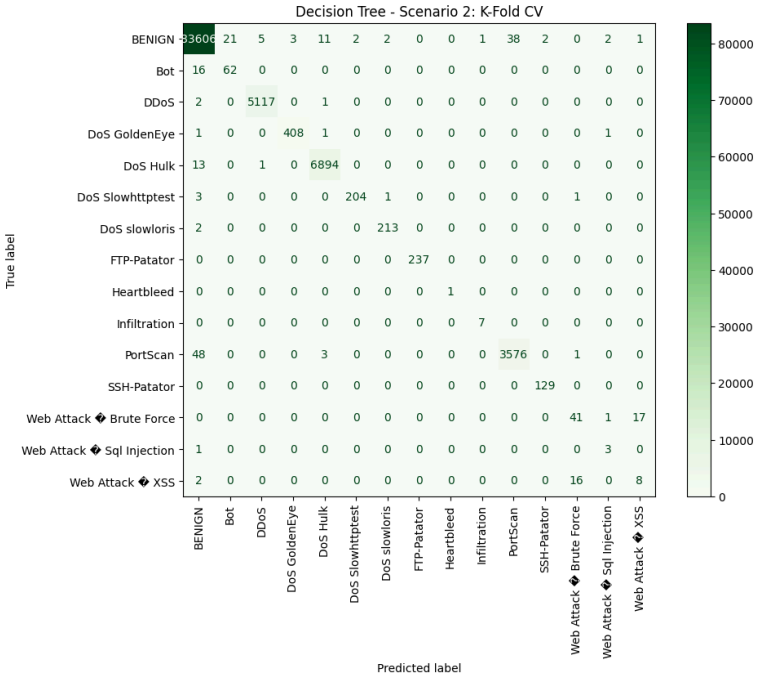
                   BENIGN       0.96      0.99      0.97     83694
                      Bot       0.94      0.37      0.53        78
                     DDoS       1.00      0.65      0.78      5120
            DoS GoldenEye       0.97      0.89      0.92       411
                 DoS Hulk       0.91      0.98      0.95      6908
         DoS Slowhttptest       0.90      0.21      0.33       209
            DoS slowloris       0.84      0.62      0.71       215
              FTP-Patator       0.81      1.00      0.90       237
               Heartbleed       1.00      1.00      1.00         1
             Infiltration       0.00      0.00      0.00         7
                 PortScan       0.84      0.54      0.65      3628
              SSH-Patator       0.00      0.00      0.00       129
  Web Attack  Brute Force       0.27      0.27      0.27        59
Web Attack  Sql Injection       0.00      0.00      0.00         4
          Web Attack  XSS       0.20      0.04      0.06        26

                 accuracy                           0.95    100726
                macro avg       0.64      0.50      0.54    100726
             weighted avg       0.95      0.95      0.95    100726
```

```
============================================================
EVALUATING DECISION TREE (K-Fold CV)
============================================================
```

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| BENIGN | 1.00 | 1.00 | 1.00 | 83694 |
| Bot | 0.75 | 0.79 | 0.77 | 78 |
| DDoS | 1.00 | 1.00 | 1.00 | 5120 |
| DoS GoldenEye | 0.99 | 0.99 | 0.99 | 411 |
| DoS Hulk | 1.00 | 1.00 | 1.00 | 6908 |
| DoS Slowhttptest | 0.99 | 0.98 | 0.98 | 209 |
| DoS slowloris | 0.99 | 0.99 | 0.99 | 215 |
| FTP-Patator | 1.00 | 1.00 | 1.00 | 237 |
| Heartbleed | 1.00 | 1.00 | 1.00 | 1 |
| Infiltration | 0.88 | 1.00 | 0.93 | 7 |
| PortScan | 0.99 | 0.99 | 0.99 | 3628 |
| SSH-Patator | 0.98 | 1.00 | 0.99 | 129 |
| Web Attack  Brute Force | 0.69 | 0.69 | 0.69 | 59 |
| Web Attack  Sql Injection | 0.43 | 0.75 | 0.55 | 4 |
| Web Attack  XSS | 0.31 | 0.31 | 0.31 | 26 |
| accuracy | | | 1.00 | 100726 |
| macro avg | 0.87 | 0.90 | 0.88 | 100726 |
| weighted avg | 1.00 | 1.00 | 1.00 | 100726 |



Στο απλό train/test split το μοντέλο εμφάνισε σχετικά χαμηλότερο Recall, χάνοντας μέρος των επιθέσεων. Αντίθετα, στο 5-Fold Cross Validation πέτυχε πολύ υψηλές επιδόσεις σε όλες τις μετρικές.

Η διαφορά αυτή δείχνει ότι το Decision Tree επηρεάζεται αρκετά από τη σύνθεση των δεδομένων εκπαίδευσης και εμφανίζει τάση overfitting όταν εκπαιδεύεται σε ένα μόνο split. Μέσω cross-validation το μοντέλο εκπαιδεύτηκε σε διαφορετικά υποσύνολα δεδομένων και κατάφερε να γενικεύσει καλύτερα.

### 5.3 Random Forest

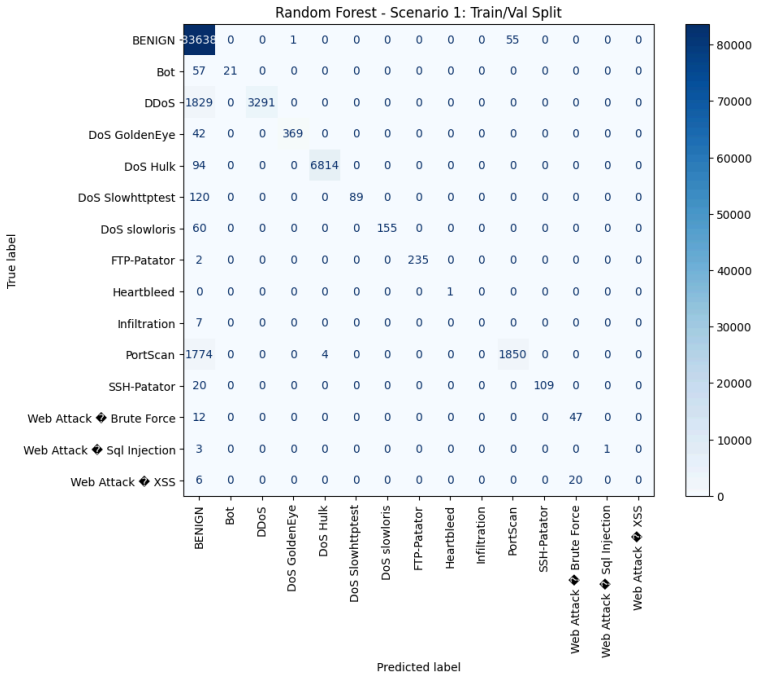
Το Random Forest παρουσίασε τη συνολικά πιο ισορροπημένη και σταθερή συμπεριφορά.

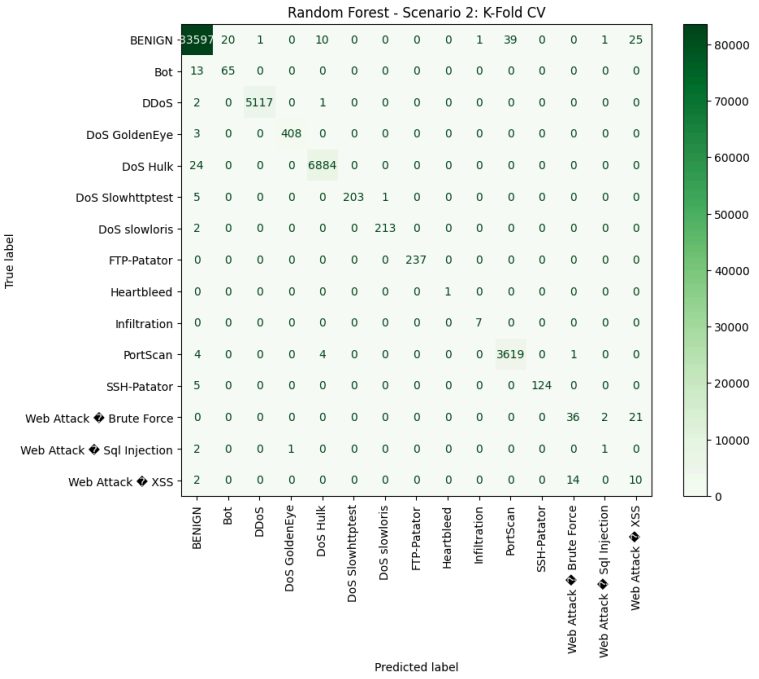
Random Forest train/val split

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| BENIGN | 0.95 | 1.00 | 0.98 | 83694 |
| Bot | 1.00 | 0.27 | 0.42 | 78 |
| DDoS | 1.00 | 0.64 | 0.78 | 5120 |
| DoS GoldenEye | 1.00 | 0.90 | 0.94 | 411 |
| DoS Hulk | 1.00 | 0.99 | 0.99 | 6908 |
| DoS Slowhttptest | 1.00 | 0.43 | 0.60 | 209 |
| DoS slowloris | 1.00 | 0.72 | 0.84 | 215 |
| FTP-Patator | 1.00 | 0.99 | 1.00 | 237 |
| Heartbleed | 1.00 | 1.00 | 1.00 | 1 |
| Infiltration | 0.00 | 0.00 | 0.00 | 7 |
| PortScan | 0.97 | 0.51 | 0.67 | 3628 |
| SSH-Patator | 1.00 | 0.84 | 0.92 | 129 |
| Web Attack  Brute Force | 0.70 | 0.80 | 0.75 | 59 |
| Web Attack  Sql Injection | 1.00 | 0.25 | 0.40 | 4 |
| Web Attack  XSS | 0.00 | 0.00 | 0.00 | 26 |
| accuracy | | | 0.96 | 100726 |
| macro avg | 0.84 | 0.62 | 0.69 | 100726 |
| weighted avg | 0.96 | 0.96 | 0.95 | 100726 |

```
============================================================
RANDOM FOREST - EVALUATING SCENARIO 2 (K-Fold CV)
============================================================
```

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| BENIGN | 1.00 | 1.00 | 1.00 | 83694 |
| Bot | 0.76 | 0.83 | 0.80 | 78 |
| DDoS | 1.00 | 1.00 | 1.00 | 5120 |
| DoS GoldenEye | 1.00 | 0.99 | 1.00 | 411 |
| DoS Hulk | 1.00 | 1.00 | 1.00 | 6908 |
| DoS Slowhttptest | 1.00 | 0.97 | 0.99 | 209 |
| DoS slowloris | 1.00 | 0.99 | 0.99 | 215 |
| FTP-Patator | 1.00 | 1.00 | 1.00 | 237 |
| Heartbleed | 1.00 | 1.00 | 1.00 | 1 |
| Infiltration | 0.88 | 1.00 | 0.93 | 7 |
| PortScan | 0.99 | 1.00 | 0.99 | 3628 |
| SSH-Patator | 1.00 | 0.96 | 0.98 | 129 |
| Web Attack  Brute Force | 0.71 | 0.61 | 0.65 | 59 |
| Web Attack  Sql Injection | 0.25 | 0.25 | 0.25 | 4 |
| Web Attack  XSS | 0.18 | 0.38 | 0.24 | 26 |
| accuracy | | | 1.00 | 100726 |
| macro avg | 0.85 | 0.87 | 0.85 | 100726 |
| weighted avg | 1.00 | 1.00 | 1.00 | 100726 |



Το μοντέλο πέτυχε υψηλό Precision και στις δύο περιπτώσεις, γεγονός που δείχνει ότι οι προβλέψεις του ήταν αρκετά αξιόπιστες και με περιορισμένα false positives. Παράλληλα, μέσω cross-validation βελτιώθηκε σημαντικά και το Recall, οδηγώντας στο υψηλότερο συνολικό F1-score μετά το Decision Tree.

Η καλύτερη συμπεριφορά του Random Forest οφείλεται στη χρήση πολλαπλών δέντρων απόφασης (ensemble learning), τα οποία μειώνουν το overfitting ενός μεμονωμένου δέντρου και οδηγούν σε καλύτερη γενίκευση στα δεδομένα.

### 5.4 Συγκριτική Ανάλυση

Συνολικά, το Logistic Regression εμφάνισε τη χαμηλότερη απόδοση, καθώς η γραμμική του προσέγγιση δεν κατάφερε να αποτυπώσει αποτελεσματικά τη σύνθετη και μη γραμμική φύση των δεδομένων. Παρότι εντόπιζε μεγάλο ποσοστό επιθέσεων (υψηλό Recall), παρήγαγε μεγάλο αριθμό λανθασμένων προβλέψεων, μειώνοντας σημαντικά το Precision και το F1-score.

Το Decision Tree παρουσίασε πολύ καλή απόδοση στο cross-validation, αλλά τα αποτελέσματά του μεταβάλλονταν σημαντικά ανάλογα με το split των δεδομένων, γεγονός που δείχνει αυξημένη ευαισθησία και πιθανότητα overfitting.

Το Random Forest ήταν ο πιο αξιόπιστος και σταθερός αλγόριθμος συνολικά. Συνδύασε υψηλό Precision, υψηλό Recall και καλύτερη γενίκευση, αξιοποιώντας την ensemble προσέγγιση πολλαπλών δέντρων. Για τον λόγο αυτό κρίθηκε ως το καταλληλότερο μοντέλο για το συγκεκριμένο πρόβλημα ανίχνευσης κυβερνοεπιθέσεων.

# Μέρος 3ο: Clustering

---

# 1. Data Preprocessing :

Όπως αναφέρθηκε και προηγουμένως χρησιμοποιήθηκε το Clustering Dataset (4% sampling) οπου  φορτώθηκε μέσω του αρχείου network_intrusion_clustering.parquet, το οποίο περιείχε 20.206 εγγραφές με 25 χαρακτηριστικά. Η στήλη Label αφαιρέθηκε από τον πίνακα χαρακτηριστικών, καθώς το clustering είναι μη επιβλεπόμενη μέθοδος ενώ διατηρήθηκε ως σειρά y για μεταγενέστερη αξιολόγηση των αποτελεσμάτων.

Εφαρμόστηκε κανονικοποίηση μέσω **StandardScaler** ώστε όλα τα χαρακτηριστικά να έχουν **μηδενική μέση τιμή και μοναδιαία τυπική απόκλιση**, απαραίτητο για αλγορίθμους που βασίζονται σε αποστάσεις (K-Means, DBSCAN, Hierarchical).

# 2. Υλοποίηση Αλγορίθμων Clustering

Υλοποιήθηκαν τρεις αλγόριθμοι clustering: **K-Means, Ιεραρχική Συσταδοποίηση (Agglomerative Hierarchical Clustering) και DBSCAN**.

Κάθε αλγόριθμος αξιολογήθηκε με τις εσωτερικές μετρικές **Silhouette Score** και **Davies-Bouldin Index**, ενώ η σύγκριση με τις αληθινές ετικέτες πραγματοποιήθηκε μέσω των μετρικών **Adjusted Rand Index (ARI), Normalized Mutual Information (NMI)** και **Homogeneity**

## K-Means

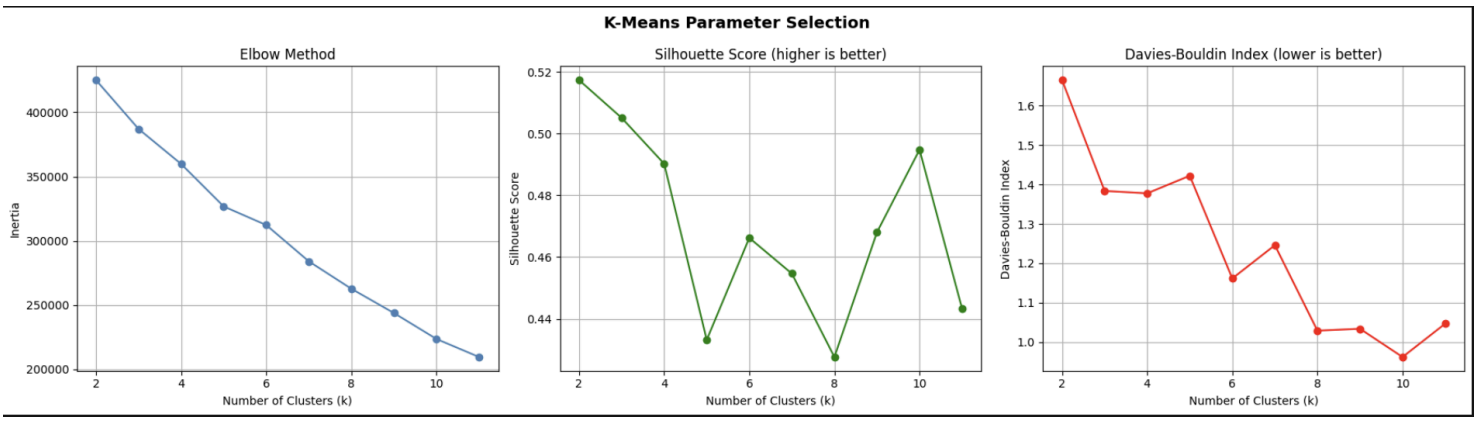
Για την επιλογή του βέλτιστου αριθμού clusters εξετάστηκε το εύρος k = 2 έως 11, αξιολογώντας τρεις δείκτες:

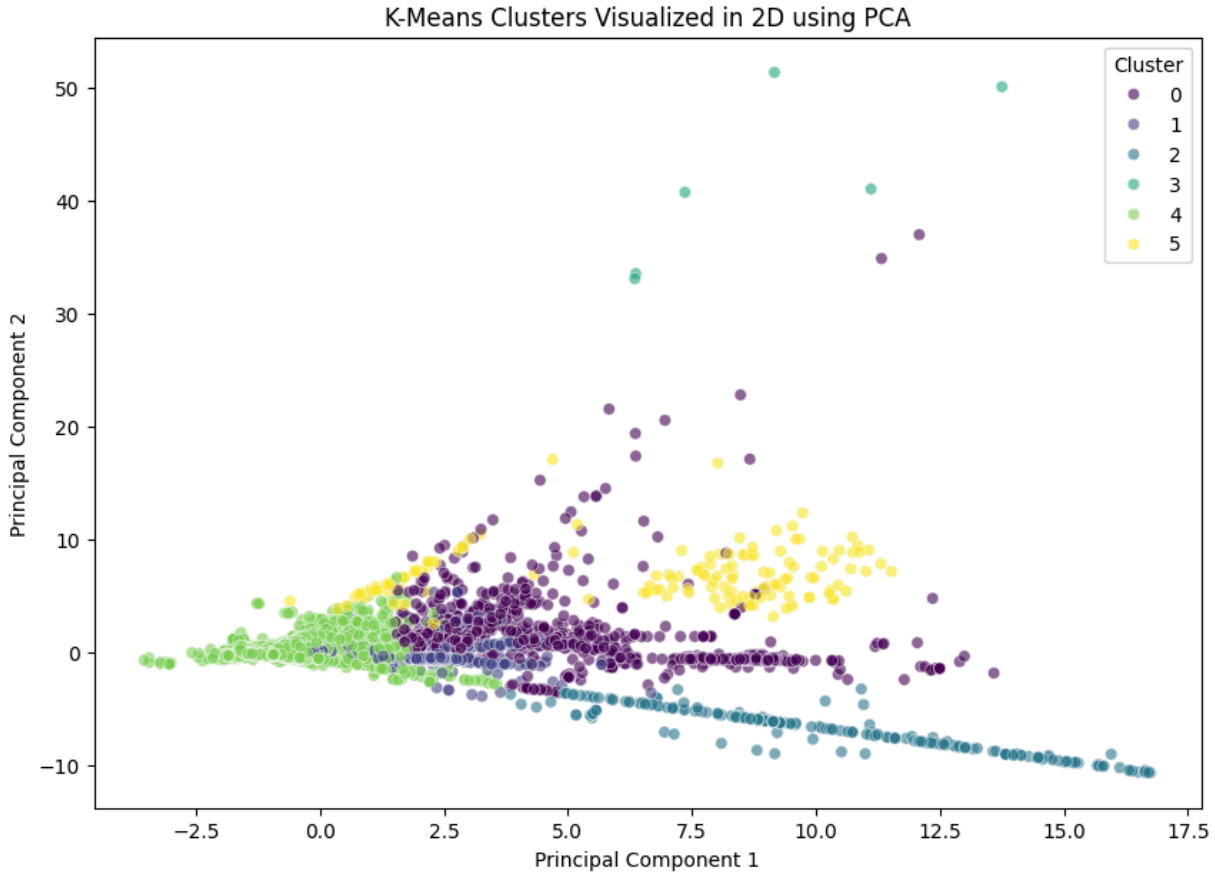
- **Elbow Method (Inertia):** Εντοπισμός του σημείου κάμψης της καμπύλης inertia.
- **Silhouette Score:** Επιλογή του k με την υψηλότερη τιμή (υψηλότερο = καλύτερο).
- **Davies-Bouldin Index:** Επιλογή του k με τη χαμηλότερη τιμή (χαμηλότερο = καλύτερο).

Η αυτοματοποιημένη αναζήτηση επέλεξε k=2, το οποίο οδηγούσε σε ένα cluster που περιείχε σχεδόν όλα τα samples και ένα με πολύ λίγα benign. Το αποτέλεσμα αυτό **δεν ήταν πληροφοριακά χρήσιμο**, καθώς δεν διέκρινε τους επιμέρους τύπους επιθέσεων. Για τον λόγο αυτό επιλέχθηκε χειροκίνητα **k=6**, το οποίο συνδύαζε ικανοποιητικό Davies-Bouldin Index με λογικό αριθμό clusters σε σχέση με τις κατηγορίες του dataset.

Το τελικό μοντέλο εκπαιδεύτηκε με KMeans(n_clusters=6, random_state=42, n_init=10) και τα labels αποθηκεύτηκαν στη στήλη KMeans_Cluster του DataFrame.

Για την οπτικοποίηση των clusters εφαρμόστηκε PCA με 2 συνιστώσες, μειώνοντας τις 25 διαστάσεις σε 2, και δημιουργήθηκε scatter plot όπου κάθε σημείο χρωματίστηκε βάσει του cluster που ανήκει. Επιπλέον, δημιουργήθηκε πίνακας cross-tabulation μεταξύ των αληθινών ετικετών (Label_Name) και των clusters του K-Means.





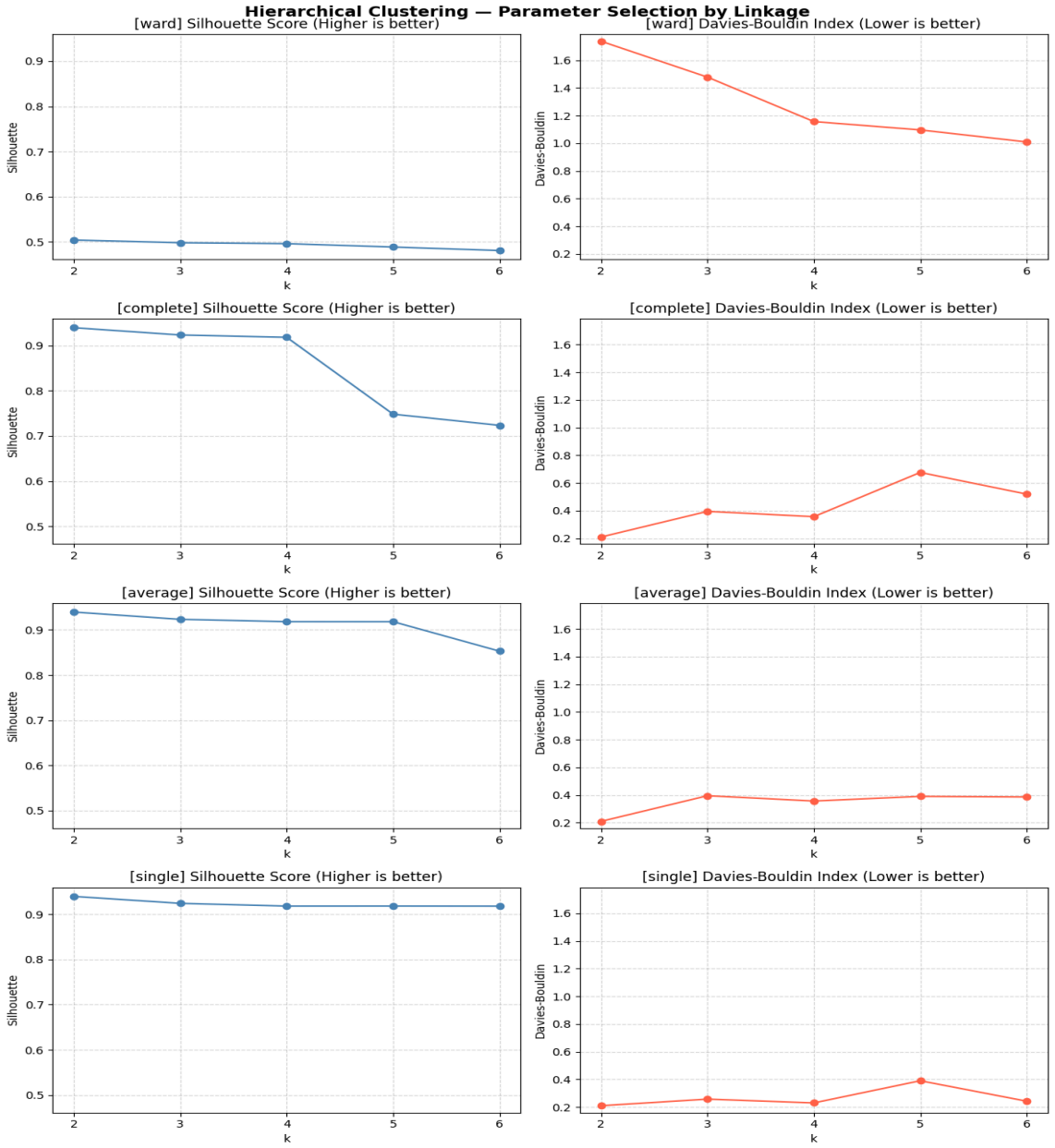

| KMeans_Cluster | 0 | 1 | 2 | 3 | 4 | 5 | All |
|---|---|---|---|---|---|---|---|
| Label_Name | | | | | | | |
| BENIGN | 1408 | 35 | 223 | 0 | 14915 | 158 | 16739 |
| Bot | 0 | 0 | 0 | 0 | 16 | 0 | 16 |
| DDoS | 0 | 351 | 0 | 0 | 673 | 0 | 1024 |
| DoS GoldenEye | 1 | 26 | 12 | 0 | 43 | 0 | 82 |
| DoS Hulk | 350 | 770 | 1 | 0 | 260 | 0 | 1381 |
| DoS Slowhttptest | 12 | 17 | 0 | 0 | 12 | 1 | 42 |
| DoS slowloris | 17 | 0 | 10 | 0 | 16 | 0 | 43 |
| FTP-Patator | 0 | 0 | 0 | 0 | 47 | 0 | 47 |
| Heartbleed | 6 | 1 | 0 | 0 | 0 | 0 | 7 |
| Infiltration | 24 | 0 | 1 | 6 | 4 | 0 | 35 |
| PortScan | 1 | 0 | 0 | 0 | 725 | 0 | 726 |
| SSH-Patator | 0 | 0 | 0 | 0 | 26 | 0 | 26 |
| Web Attack  Brute Force | 0 | 0 | 0 | 0 | 12 | 0 | 12 |
| Web Attack  Sql Injection | 0 | 0 | 0 | 0 | 21 | 0 | 21 |
| Web Attack  XSS | 0 | 0 | 0 | 0 | 5 | 0 | 5 |
| All | 1819 | 1200 | 247 | 6 | 16775 | 159 | 20206 |

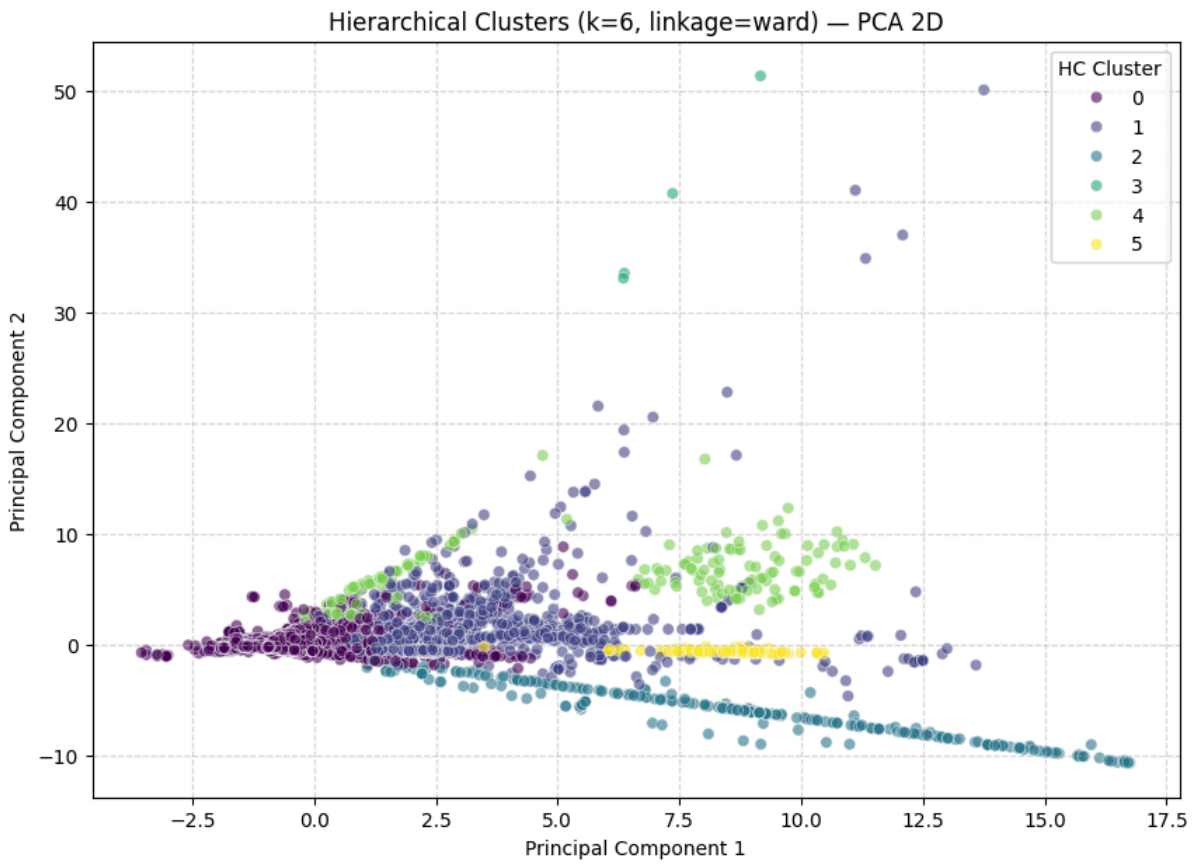
## Ιεραρχική Συσταδοποίηση (Hierarchical Clustering)

Για την επιλογή των βέλτιστων παραμέτρων εξετάστηκαν τέσσερις μέθοδοι σύνδεσης (linkage) **ward**, **complete**, **average** και **single** , σε συνδυασμό με εύρος k = 2 έως 10.

Για κάθε συνδυασμό υπολογίστηκαν το Silhouette Score, το Davies-Bouldin Index και το ποσοστό εγγραφών του μικρότερου cluster (min_cluster_pct). Το τελευταίο κριτήριο προστέθηκε ώστε να αποκλειστούν ρυθμίσεις που παρήγαγαν degenerate clusters, δηλαδή clusters με λιγότερο από 5% των δεδομένων — κάτι ιδιαίτερα σύνηθες με τη μέθοδο single λόγω του chaining effect.

Βάσει της ανάλυσης, επιλέχθηκε αυτόματα ο συνδυασμός με k=2 και linkage='single'. Παρόλα αυτά με βάση τα παρακάτω διαγράμματα **έγινε χειροκίνητη η επιλογή του k=3 και ward για linkage** καθώς παρέχει πιο ουσιαστικό διαχωρισμό των clusters. Το τελικό μοντέλο εκπαιδεύτηκε με AgglomerativeClustering(metric='euclidean') και τα labels αποθηκεύτηκαν στη στήλη HC_Cluster. Ακολούθησε οπτικοποίηση μέσω PCA 2D και cross-tabulation με τις αληθινές ετικέτες.






Hierarchical Cluster Distribution vs. True Label Names:

| HC_Cluster | 0 | 1 | 2 | 3 | 4 | 5 | All |
|---|---|---|---|---|---|---|---|
| Label_Name | | | | | | | |
| BENIGN | 14709 | 1554 | 271 | 0 | 205 | 0 | 16739 |
| Bot | 16 | 0 | 0 | 0 | 0 | 0 | 16 |
| DDoS | 857 | 167 | 0 | 0 | 0 | 0 | 1024 |
| DoS GoldenEye | 60 | 2 | 20 | 0 | 0 | 0 | 82 |
| DoS Hulk | 313 | 721 | 0 | 0 | 0 | 347 | 1381 |
| DoS Slowhttptest | 13 | 29 | 0 | 0 | 0 | 0 | 42 |
| DoS slowloris | 16 | 17 | 10 | 0 | 0 | 0 | 43 |
| FTP-Patator | 47 | 0 | 0 | 0 | 0 | 0 | 47 |
| Heartbleed | 0 | 7 | 0 | 0 | 0 | 0 | 7 |
| Infiltration | 4 | 26 | 1 | 4 | 0 | 0 | 35 |
| PortScan | 725 | 0 | 1 | 0 | 0 | 0 | 726 |
| SSH-Patator | 26 | 0 | 0 | 0 | 0 | 0 | 26 |
| Web Attack  Brute Force | 12 | 0 | 0 | 0 | 0 | 0 | 12 |
| Web Attack  Sql Injection | 21 | 0 | 0 | 0 | 0 | 0 | 21 |
| Web Attack  XSS | 5 | 0 | 0 | 0 | 0 | 0 | 5 |
| All | 16824 | 2523 | 303 | 4 | 205 | 347 | 20206 |

## DBSCAN

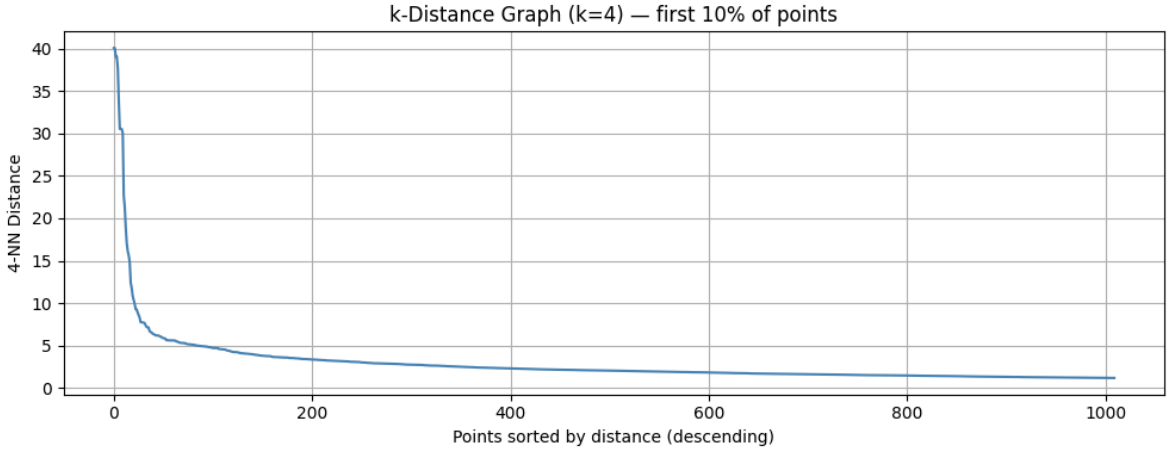
Ο αλγόριθμος DBSCAN απαιτεί τη ρύθμιση δύο παραμέτρων: eps (ακτίνα γειτονιάς) και min_samples(ελάχιστος αριθμός σημείων για σχηματισμό cluster). Για τον προσδιορισμό κατάλληλης τιμής eps δημιουργήθηκε **k-Distance Graph** με k=4 γείτονες. Οι αποστάσεις ταξινομήθηκαν φθίνουσα και αποτυπώθηκε το πρώτο 10% των σημείων, ώστε να εντοπιστεί το σημείο κάμψης (elbow) που υποδηλώνει κατάλληλη τιμή eps.

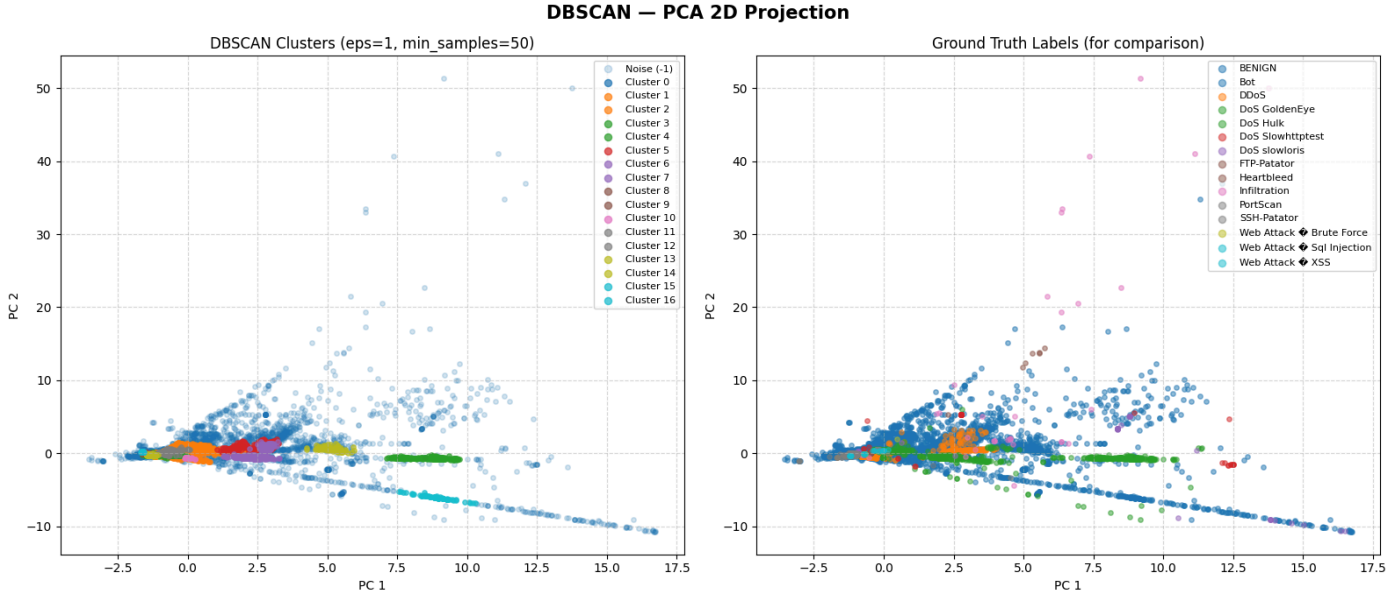
Στη συνέχεια πραγματοποιήθηκε **Grid Search** για τον εντοπισμό της βέλτιστης παραμέτρων, εξετάζοντας:

- **eps:** 0.5, 0.8, 1.0, 1.3, 1.5, 2.0, 2.5
- **min_samples:** 5, 10, 15, 20, 50, 100

Για κάθε συνδυασμό που παρήγαγε τουλάχιστον 2 clusters υπολογίστηκε το Silhouette Score, και επιλέχθηκε ο συνδυασμός με την υψηλότερη τιμή. Το τελικό μοντέλο εκτυπώθηκε με eps = 1 και min_samples = 50 ως καλύτερες παραμέτρους.

Για την οπτικοποίηση δημιουργήθηκε διπλό PCA scatter plot: αριστερά τα clusters του DBSCAN (με τα noise points να απεικονίζονται με χαμηλή διαφάνεια) και δεξιά οι αληθινές ετικέτες για σύγκριση.



| DBSCAN_Cluster<br>Label_Name | -1 | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | All |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| BENIGN | 2263 | 590 | 10108 | 507 | 0 | 1386 | 390 | 0 | 227 | 216 | 108 | 104 | 90 | 157 | 368 | 94 | 79 | 52 | 16739 |
| Bot | 1 | 0 | 9 | 0 | 0 | 6 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 16 |
| DDoS | 162 | 0 | 851 | 0 | 0 | 0 | 0 | 11 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1024 |
| DoS GoldenEye | 43 | 0 | 39 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 82 |
| DoS Hulk | 104 | 0 | 192 | 0 | 319 | 1 | 0 | 695 | 0 | 0 | 0 | 69 | 0 | 0 | 1 | 0 | 0 | 0 | 1381 |
| DoS Slowhttptest | 38 | 0 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 42 |
| DoS slowloris | 31 | 1 | 11 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 43 |
| FTP-Patator | 0 | 17 | 30 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 47 |
| Heartbleed | 7 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 7 |
| Infiltration | 31 | 0 | 3 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 35 |
| PortScan | 27 | 0 | 694 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 3 | 2 | 0 | 0 | 0 | 0 | 726 |
| SSH-Patator | 0 | 1 | 23 | 0 | 0 | 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 26 |
| Web Attack  Brute Force | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 11 | 0 | 0 | 0 | 0 | 0 | 12 |
| Web Attack  Sql Injection | 0 | 0 | 12 | 0 | 0 | 9 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 21 |
| Web Attack  XSS | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 5 | 0 | 0 | 0 | 0 | 0 | 5 |
| All | 2708 | 609 | 11972 | 507 | 319 | 1407 | 390 | 706 | 227 | 216 | 108 | 173 | 111 | 159 | 369 | 94 | 79 | 52 | 20206 |

# 3. Αξιολόγηση και Συγκριτική Ανάλυση

Η τελική σύγκριση των τριών αλγορίθμων πραγματοποιήθηκε με βάση τρεις μετρικές αξιολόγησης έναντι των αληθινών ετικετών:

- **Adjusted Rand Index (ARI):** Μετρά την ομοιότητα μεταξύ της πρόβλεψης και των αληθινών ετικετών, με διόρθωση για τυχαία συμφωνία (1.0 = τέλεια αντιστοίχιση, 0 = τυχαία).
- **Normalized Mutual Information (NMI):** Μετρά πόση πληροφορία μοιράζονται οι προβλεπόμενες και οι αληθινές ετικέτες (1.0 = πλήρης αντιστοίχιση, 0 = καμία).
- **Homogeneity:** Μετρά αν κάθε cluster περιέχει μόνο σημεία από μία κλάση.

| | Algorithm | Adjusted Rand Index (ARI) | Normalized Mutual Info (NMI) | Homogeneity |
|---|---|---|---|---|
| 0 | K-Means | 0.319703 | 0.251393 | 0.237195 |
| 1 | Hierarchical | 0.217726 | 0.138084 | 0.112723 |
| 2 | DBSCAN | 0.043858 | 0.188782 | 0.304787 |

**K-Means:** Ο αλγόριθμος **K-Means** παρουσίασε την καλύτερη συνολική απόδοση, επιτυγχάνοντας τις υψηλότερες τιμές σε ARI (0.3197) και NMI (0.2514). Τα αποτελέσματα δείχνουν ότι κατάφερε να αποτυπώσει σε μεγαλύτερο βαθμό τη δομή των πραγματικών κατηγοριών του dataset.

**Hierarchical Clustering:** Το Hierarchical clustering παρουσίασε χαμηλότερες επιδόσεις σε όλες τις μετρικές (ARI = 0.2177, NMI = 0.1381, Homogeneity = 0.1127), γεγονός που υποδηλώνει μικρότερη συμφωνία με τις πραγματικές κατηγορίες. Παρότι η μέθοδος ward παρήγαγε πιο ισορροπημένα clusters, η συνολική ποιότητα συσταδοποίησης παρέμεινε χαμηλότερη από αυτή του K-Means.

**DBSCAN:** Ο DBSCAN εμφάνισε τη χαμηλότερη συνολική απόδοση ως προς το ARI (0.0439), γεγονός που δείχνει περιορισμένη συμφωνία μεταξύ των clusters και των πραγματικών labels. Παρόλα αυτά, παρουσίασε τη μεγαλύτερη τιμή Homogeneity (0.3048). Η επίδοση αυτή οφείλεται κυρίως στο ότι αρκετά clusters περιείχαν σχεδόν αποκλειστικά δείγματα της κλάσης BENIGN, καθώς και στην απομόνωση μεγάλου αριθμού δειγμάτων ως noise (-1).

Από την ανάλυση των clusters παρατηρήθηκε ότι το μεγαλύτερο cluster κυριαρχείται από BENIGN, ενώ σημαντικό μέρος των επιθέσεων είτε ενσωματώνεται σε αυτό είτε χαρακτηρίζεται ως outlier. Παράλληλα, ορισμένες επιθέσεις, όπως τμήματα του DoS Hulk, σχημάτισαν πιο “καθαρά” και διακριτά clusters.

Συνολικά, το **K-Means** κρίθηκε ως ο πιο πρακτικός και ερμηνεύσιμος αλγόριθμος για το συγκεκριμένο dataset, κυρίως λόγω της ευκολίας ελέγχου του αριθμού clusters και της σταθερής απόδοσής του. Η **Ιεραρχική Συσταδοποίηση** με linkage ward αποτέλεσε αξιόπιστη εναλλακτική, ενώ ο **DBSCAN** αντιμετώπισε δομικές δυσκολίες λόγω της φύσης του dataset (ακραία ανισορροπία κλάσεων και διαφορετικές πυκνότητες).

# Περιβάλλον υλοποίησης

---

### 1. Αναλυτική Καταγραφή Περιβάλλοντος Υλοποίησης

Για την υλοποίηση του συγκεκριμένου project υιοθετήθηκε μια υβριδική προσέγγιση όσον αφορά τα περιβάλλοντα ανάπτυξης:

- **Cloud Περιβάλλον (Kaggle):** Η Διερευνητική Ανάλυση Δεδομένων (EDA) υλοποιήθηκε στο διαδικτυακό περιβάλλον του Kaggle μέσω του αρχείου eda.ipynb. Το περιβάλλον αυτό προσέφερε άμεση πρόσβαση στο αρχικό dataset μέσω του API kagglehub και παρείχε περισσότερους πόρους για την πιο εύκολη επεξεργασία ολόκληρου του dataset.
- **Τοπικό Περιβάλλον (Local Virtual Environment):** Οι διαδικασίες προεπεξεργασίας (preprocessing.ipynb), ομαδοποίησης (clustering.ipynb) και ταξινόμησης (classification.ipynb) εκτελέστηκαν τοπικά. Για το σκοπό αυτό, δημιουργήθηκε ένα απομονωμένο εικονικό περιβάλλον (Virtual Environment - venv) το οποίο βασίστηκε στην έκδοση **Python 3.14**.

### 2. Βιβλιοθήκες Λογισμικού

Για την ολοκλήρωση των διαφόρων βημάτων του project χρησιμοποιήθηκαν οι εξής εξωτερικές και ενσωματωμένες βιβλιοθήκες της Python:

- **Διαχείριση και Επεξεργασία Δεδομένων:**
  - **Pandas:** Χρησιμοποιήθηκε εκτενώς σε όλα τα στάδια (EDA, Preprocessing, Clustering, Classification) για την ανάγνωση, συγχώνευση (concat), διαχείριση (DataFrames) και αποθήκευση των δεδομένων σε αποδοτική μορφή Parquet.
  - **NumPy:** Αξιοποιήθηκε για μαθηματικούς υπολογισμούς, διαχείριση πινάκων, και σύνθετες δομές δεδομένων (όπως η συνένωση πινάκων με np.vstack / np.concatenate).
- **Μηχανική Μάθηση & Ομαδοποίηση:**
  - **Scikit-Learn (sklearn):** Αποτέλεσε τη βασική βιβλιοθήκη για την υλοποίηση των αλγορίθμων. Χρησιμοποιήθηκαν τα παρακάτω αρθρώματα:
    - *Προεπεξεργασία & Διαχωρισμός:* StandardScaler (για την κανονικοποίηση), LabelEncoder, train_test_split, StratifiedKFold, PredefinedSplit και GridSearchCV (για την εύρεση βέλτιστων υπερπαραμέτρων).
    - *Μοντέλα Ταξινόμησης:* LogisticRegression, DecisionTreeClassifier, RandomForestClassifier.
    - *Μοντέλα Ομαδοποίησης:* KMeans, AgglomerativeClustering (Ιεραρχική), DBSCAN.
    - *Μετρικές Αξιολόγησης:* silhouette_score, davies_bouldin_score, και διάφορες μετρικές αξιολόγησης ως προς το Label (όπως Adjusted Rand Index).
    - *Pipelines:* Pipeline για την αυτοματοποίηση των βημάτων μετασχηματισμού και εκπαίδευσης.
- **Οπτικοποίηση Δεδομένων:**
  - **Matplotlib (matplotlib.pyplot):** Για τη δημιουργία γραφημάτων απεικόνισης της δικτυακής κίνησης και των clusters.
  - **Seaborn:** Για την κατασκευή πιο σύνθετων στατιστικών γραφημάτων, όπως τα Heatmaps συσχέτισης (Correlation Heatmaps) μεταξύ των χαρακτηριστικών.
- **Εισαγωγή Δεδομένων & Σύστημα:**
  - **Kagglehub:** Για την απευθείας λήψη του συνόλου δεδομένων μέσα από το Kaggle.
  - **Standard Library της Python:** os (διαχείριση συστήματος), json (αποθήκευση/ανάγνωση των label mappings), warnings (για απόκρυψη προειδοποιήσεων) και itertools (για παραγωγή συνδυασμών στο grid search του DBSCAN).

### 3. Διαδικασία Εγκατάστασης Τοπικού Περιβάλλοντος

Για την αναπαραγωγή των πειραμάτων και την ορθή εκτέλεση των κωδίκων (preprocessing.ipynb, clustering.ipynb, classification.ipynb) στο τοπικό μηχάνημα, απαιτούνται τα ακόλουθα βήματα μέσω τερματικού (Terminal / Command Prompt):

1. **Έλεγχος Έκδοσης Python:** Βεβαιωθείτε ότι υπάρχει εγκατεστημένη η έκδοση **Python 3.14** στο σύστημά σας.
2. **Δημιουργία Virtual Environment:** Προτείνεται η δημιουργία ενός απομονωμένου περιβάλλοντος με το όνομα venv: python -m venv venv
3. **Ενεργοποίηση του Virtual Environment:** source venv/bin/activate

4. **Εγκατάσταση των απαραίτητων βιβλιοθηκών:** Εγκατάσταση των πακέτων λογισμικού που απαιτούνται, συμπεριλαμβανομένου και του μηχανισμού ανάγνωσης αρχείων Parquet (pyarrow ή fastparquet): pip install pandas numpy scikit-learn matplotlib seaborn jupyter pyarrow